

BIG DATA & HADOOP

YARN and Hive

MODULE 1: YARN Architecture

1.1 What is YARN?

YARN stands for Yet Another Resource Negotiator. Introduced in Hadoop 2.0 (2013), it was a major architectural breakthrough that separated resource management from processing logic, allowing Hadoop to support multiple processing frameworks simultaneously.

Key Innovation

Hadoop 1.x had a single JobTracker that handled both resource management AND job scheduling. YARN splits these into separate, dedicated components — making the system far more scalable and flexible.

1.2 Why Was YARN Created? — Problems with Hadoop 1.x

Problem	Description
Scalability Bottleneck	Limited to ~4,000 nodes. The JobTracker became overwhelmed handling everything alone.
Single Point of Failure	If the JobTracker crashed, the entire cluster stopped — no fault tolerance.
Resource Wastage	Fixed map and reduce slots couldn't be shared or reused across task types.
Only MapReduce Supported	No way to run Spark, Flink, or any other processing framework.
Tight Coupling	Resource management and job scheduling were bundled together — hard to maintain or extend.

1.3 YARN Core Components

Component 1: Resource Manager (RM)

Location: Runs on the master node (one per cluster). It is the global authority for resource management across the entire cluster.

The Resource Manager has two internal sub-components:

Sub-Component	Role
---------------	------

Scheduler	Allocates resources (CPU, memory) to applications. It is a pure scheduler — it does NOT track task status or restart failed tasks. Supports pluggable scheduling policies.
Application Manager (ASM)	Accepts job submissions, negotiates the first container for the Application Master, monitors AMs, and restarts them on failure.

Teacher Note

Emphasize: The Scheduler only allocates resources. It does not monitor tasks. The Application Master is responsible for monitoring individual tasks.

Component 2: Node Manager (NM)

Location: Runs on every worker node (one per node). It is the per-node agent responsible for managing local resources and containers.

Node Manager Responsibilities
Launch and monitor containers on the local node
Manage local resources: CPU, memory, disk, network
Send heartbeats to the Resource Manager every few seconds
Report resource availability and container status
Enforce resource limits on running containers
Handle the full container lifecycle (start to finish)

Component 3: Application Master (AM)

One Application Master is created per job/application. It runs inside a container on a worker node and is responsible for managing that specific application's entire lifecycle.

Responsibility	What it Does
Resource Negotiation	Requests containers from the Resource Manager, specifying CPU and memory requirements.
Task Management	Launches tasks in containers, monitors progress, handles failures, and retries.
Progress Tracking	Aggregates and reports progress to the Resource Manager and clients.
Fault Tolerance	Detects task failures and requests replacement containers automatically.

Examples of Application Masters: MRAppMaster (MapReduce), Spark AM, Flink AM, Tez AM.

Component 4: Container

A container is the basic unit of resource allocation in YARN. It is an abstract bundle of resources (CPU + Memory) granted by the Resource Manager and managed by the Node Manager.

Property	Details
Contents	CPU cores (vCores) + Memory (RAM in MB/GB) + Disk + Network
Lifecycle	Request → Allocate → Launch → Execute → Complete → Release
Scope	Runs a single task or process
Duration	Short-lived — exists only for the duration of the task

MODULE 2: YARN Workflow & Execution

2.1 Complete Application Execution — Step by Step

Phase 1: Application Submission

Step 1	Client submits the application to the Resource Manager. This includes job configuration, input/output paths, and JAR files.
Step 2	The Resource Manager's Application Manager receives the submission and validates it.
Step 3	Application Manager negotiates a container with the Scheduler for the Application Master.
Step 4	Resource Manager allocates Container_0 specifically to run the Application Master.
Step 5	Resource Manager contacts a suitable Node Manager to launch the Application Master inside Container_0.

Phase 2: Resource Negotiation

Step 1	Application Master starts, registers with the Resource Manager, and provides a tracking URL.
Step 2	AM analyzes the job requirements and calculates how many containers it needs (size + count).
Step 3	AM sends a resource request to the Resource Manager, specifying vCores, memory, and optional locality preferences.
Step 4	Scheduler finds available resources on Node Managers that match the request.
Step 5	Resource Manager allocates containers and sends the allocation response (with tokens) back to the AM.

Phase 3: Task Execution

Step 1	AM contacts the appropriate Node Managers with container tokens to launch tasks.
Step 2	Node Managers start the containers and launch the assigned tasks inside them.
Step 3	Tasks begin executing. They send progress updates back to the Application Master periodically.

Step 4	AM monitors all task progress and reports overall job status to the Resource Manager.
---------------	---

Phase 4: Completion & Cleanup

Step 1	All tasks finish and report completion to the Application Master.
Step 2	AM verifies all tasks have completed successfully.
Step 3	AM reports final job status to the Resource Manager and unregisters from it.
Step 4	All containers are released back to the cluster resource pool. Resources are now available for the next job.

MODULE 3: YARN Scheduling Policies

3.1 Overview of Scheduling

The YARN Scheduler is responsible for allocating resources to applications. YARN supports three pluggable scheduling policies. The right choice depends on the cluster's use case, number of users, and required guarantees.

3.2 The Three Scheduling Policies

Policy 1: FIFO Scheduler (First In, First Out)

Aspect	Details
How it works	Applications are queued and processed in order of arrival. The first job gets ALL cluster resources. The next job waits until the first finishes.
Strengths	Simple to understand and configure. No setup overhead.
Weaknesses	No fairness. Short jobs wait behind long jobs. Unsuitable for shared clusters.
Best used for	Single-user or single-workload clusters. Development environments.

Policy 2: Capacity Scheduler

Multiple queues are defined, each with a guaranteed capacity percentage. For example: Production Queue (70%), Development Queue (30%). Each queue gets its minimum share, but can borrow unused capacity from other queues.

Aspect	Details
Queue Structure	Hierarchical — root queue splits into child queues, each with capacity guarantees.
Elasticity	Queues can borrow unused capacity from sibling queues dynamically.
Strengths	Multi-tenant support, SLA guarantees, resource isolation between teams.
Best used for	Enterprise clusters shared by multiple teams or departments with defined SLAs.

Policy 3: Fair Scheduler

Resources are shared equally across all running applications over time. When only one job is running, it gets everything. When more jobs arrive, resources are dynamically redistributed so all jobs get a fair share.

Aspect	Details
Dynamic sharing	Resources reallocate as jobs arrive and complete — no reserved slots.
Preemption	Can reclaim resources from jobs that are using more than their fair share.
Strengths	No job starves — all jobs make progress simultaneously.
Best used for	Shared research or academic clusters where fairness is the top priority.

3.3 Scheduling Policy Comparison

Feature	FIFO	Capacity	Fair
Multi-tenancy	No	Yes	Yes
Queue Support	Single queue	Multiple queues	Multiple pools
Fairness	None	Per-queue	Dynamic equal share
Preemption	No	Optional	Yes
Complexity	Low	Medium	Medium-High
Best for	Simple clusters	Enterprise SLAs	Shared research clusters

MODULE 4: Introduction to Apache Hive

4.1 What is Apache Hive?

Apache Hive is a data warehouse software built on top of Hadoop. It allows analysts and engineers to read, write, and manage large datasets stored in HDFS using a familiar SQL-like query language called HiveQL (Hive Query Language).

Feature	Data Warehouse	Dataset	Data Lake
Definition	Centralized repo of structured, processed data for reporting & analytics	A collection of related data (any format, any size)	Large repo storing raw data in native format until needed
Data Structure	Highly structured (schema-on-write)	Varies (structured, semi-, unstructured)	Unstructured/semi-structured (schema-on-read)
Data Format	Tables, rows, columns (SQL)	CSV, JSON, XML, DB tables, etc.	Files, blobs, JSON, Parquet, images, video, etc.
Storage Cost	High	Low–Medium	Very Low
Processing	Pre-processed & cleaned (ETL)	Depends on source	Raw, processed on demand (ELT)
Query Speed	Very Fast	Varies	Slower (without optimization)
Users	Business analysts, BI tools	Developers, analysts, ML engineers	Data scientists, ML engineers
Use Case	Business intelligence, dashboards, reporting	Analysis, ML training, sharing data	Big data analytics, ML, long-term raw storage
Scalability	Moderate–High	Low–Moderate	Extremely High
Schema	Fixed (schema-on-write)	Flexible	Flexible (schema-on-read)
Data Quality	High (curated)	Varies	Low–Medium (raw)
Examples	Snowflake, BigQuery, Redshift	Kaggle datasets, CSV files, DB tables	AWS S3, Azure Data Lake, GCP Cloud Storage
Governance	Strong	Varies	Weaker (can become a "data swamp")

Quick model:

- **Data Warehouse** → Clean, organized library — everything catalogued and ready to read
- **Dataset** → A single book or folder — specific, self-contained collection of data
- **Data Lake** → A reservoir — dump everything in raw, filter and process it later

4.2 The Problem Hive Solves

The Core Challenge

Hadoop processes data using MapReduce — which requires writing complex Java programs. Most data analysts know SQL but not Java/MapReduce. Hive bridges this gap by letting users write SQL queries that are automatically translated into MapReduce jobs.

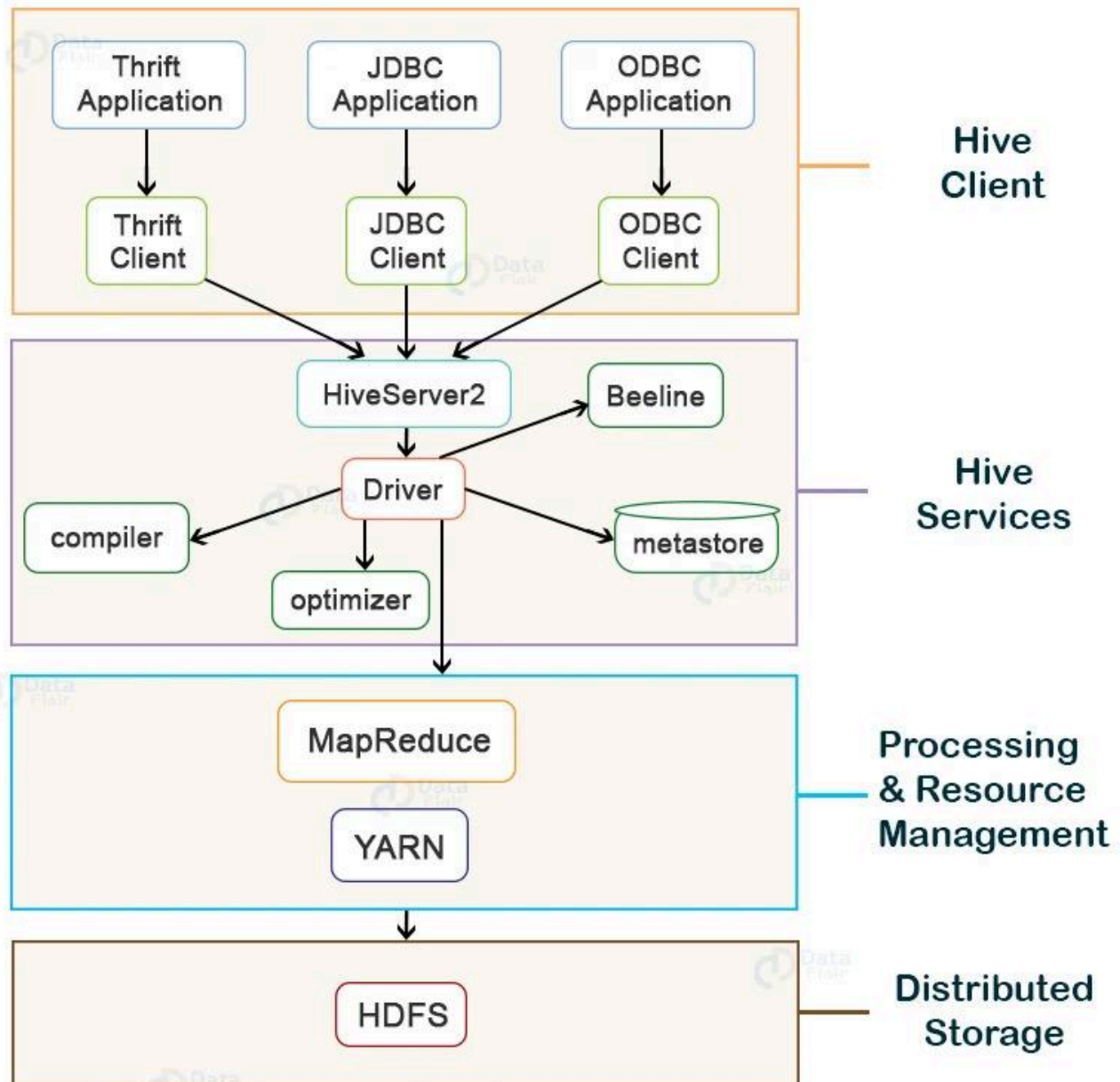
4.3 Key Characteristics of Hive

Characteristic	Explanation
Not a database	Hive is a data warehouse solution — it sits on top of HDFS, not a traditional RDBMS.
Schema on Read	Unlike traditional databases (schema on write), Hive applies the schema when data is read, not when it is stored.
Best for batch processing	Ideal for large-scale analytics, ETL jobs, and historical analysis.
NOT for real-time	High latency makes Hive unsuitable for OLTP or real-time query applications.
Multiple language support	Supports Python, Java, C++, Ruby via JDBC, ODBC, and Thrift drivers.

MODULE 5: Hive Architecture & Components

5.1 Architecture Overview

Hive's architecture is divided into three logical layers: Client Layer (how users connect), Services Layer (the query processing engine), and Storage Layer (HDFS + MapReduce).



Hive Architecture & Its Components

5.2 Layer 1 — Hive Clients

Users can connect to Hive in three ways:

Client Type	Description
Thrift Client	Based on Apache Thrift protocol. Enables cross-platform communication and supports multiple programming languages (Python, Java, C++, etc.).
JDBC Client	For Java applications. Uses Thrift underneath. Provides a familiar interface for Java developers — similar to connecting to any standard database.
ODBC Client	For applications and BI tools that use the ODBC protocol. Also uses Thrift underneath. Common in enterprise BI environments.

5.3 Layer 2 — Hive Services (The Core Engine)

Beeline

A command-line interface for submitting HiveQL queries — similar to the MySQL command-line tool. Beeline connects to HiveServer2 and is the modern replacement for the older Hive CLI.

HiveServer2

Handles incoming client requests. A key improvement over the original HiveServer — it supports multiple concurrent client connections simultaneously. Provides JDBC and ODBC interfaces.

Hive Driver

The first point of contact for every query submitted. It creates a session handle for the query, manages the lifecycle of a query from submission to results, and passes the query to the Compiler.

Hive Compiler

Parses and validates the HiveQL query. It performs semantic analysis (checking that the query makes logical sense), type-checking using metadata from the Metastore, and generates an optimized execution plan in the form of a DAG (Directed Acyclic Graph).

Optimizer

Takes the execution plan (DAG) from the Compiler and improves it. It may split tasks, rearrange stages, or eliminate redundant operations. The goal is to make queries run as fast and efficiently as possible.

Execution Engine

Takes the optimized plan and executes it by submitting jobs to Hadoop's MapReduce (or Tez/Spark). It manages the dependencies between stages — ensuring Stage 2 only starts after Stage 1 completes.

Metastore (Critical Component)

The central repository for all Hive metadata. Without the Metastore, Hive has no knowledge of tables, schemas, or data locations.

What the Metastore Stores
Table names, column names, and data types
Table structure and schema definitions
Partition information (for partitioned tables)
File locations in HDFS where data is stored
Serializer/Deserializer (SerDe) information for reading/writing data

HCatalog

A table and storage management layer built on top of the Metastore. It allows other tools like Apache Pig and MapReduce programs to access Hive tables without needing to write Hive-specific code.

WebHCat

A REST API layer on top of HCatalog. Allows HTTP-based access to metadata and job submission — enabling web applications and scripts to interact with Hive over standard HTTP requests.

5.4 Layer 3 — Processing & Storage

Layer	Description
Processing Framework	Hive translates HiveQL into MapReduce jobs (or Tez/Spark jobs in modern configurations). These jobs execute in parallel across the cluster.
Distributed Storage (HDFS)	All Hive table data is physically stored in HDFS. Hive manages the mapping between logical tables and their physical HDFS file locations through the Metastore.

MODULE 6: Hive Query Workflow

6.1 End-to-End Query Execution

Let's trace what happens when a user submits this query:

```
SELECT name, age FROM employees WHERE age > 30;
```

Step 1	USER SUBMITS QUERY — The user types the query in Beeline or a JDBC client and executes it. The interface calls the Driver's execute() method.
Step 2	DRIVER RECEIVES QUERY — The Driver creates a session handle for this query and forwards it to the Compiler for processing.
Step 3	COMPILER REQUESTS METADATA — The Compiler needs to know about the 'employees' table. It sends a metadata request to the Metastore.
Step 4	METASTORE RETURNS METADATA — The Metastore retrieves and sends back: table schema, column names and types, partition info, and HDFS file locations.
Step 5	COMPILER: SEMANTIC ANALYSIS — Compiler validates the query: checks 'age > 30' is valid (age is numeric), confirms all column names exist, ensures query logic is sound.
Step 6	COMPILER: GENERATES DAG — Compiler creates the execution plan as a Directed Acyclic Graph: Map operator trees (for filtering and reading) and Reduce operator trees (for aggregation if needed).
Step 7	OPTIMIZER IMPROVES THE PLAN — Optimizer analyzes the DAG, identifies optimization opportunities, and produces a more efficient execution plan.
Step 8	DRIVER RECEIVES OPTIMIZED PLAN — Compiler sends the final execution plan back to the Driver.
Step 9	DRIVER SENDS PLAN TO EXECUTION ENGINE — Driver forwards the plan to the Execution Engine for actual execution.
Step 10	EXECUTION ENGINE SUBMITS MapReduce JOBS — Engine breaks the DAG into stages, submits each as a MapReduce job. Deserializers read HDFS data, operators process it, serializers write temp output files.
Step 11	MapReduce JOBS EXECUTE — Mappers read employee data from HDFS and filter records where age > 30. Reducers handle any grouping/aggregation. Output stored in temporary HDFS files.
Step 12	RESULTS RETURNED TO USER — Execution Engine reads final temporary files from HDFS. Driver fetches results. Results are sent back to the Beeline/JDBC client. The user sees the output table.

Key Flow Summary

User → Driver → Compiler → Metastore → Compiler (plan) → Optimizer → Driver →
Execution Engine → MapReduce (HDFS) → Results → User

MODULE 7: DAG — Directed Acyclic Graph

7.1 What is a DAG?

A Directed Acyclic Graph (DAG) is a visual and logical representation of tasks and their dependencies. Hive's Compiler generates a DAG as the execution plan for every query. It describes which tasks need to run, in what order, and which can run in parallel.

7.2 Breaking Down the Term

Term	Meaning in Context
Graph	A structure made of nodes (tasks/stages) and edges (dependency arrows connecting them).
Directed	Arrows have direction — they show that Task B depends on Task A completing first. You can only move forward along an arrow.
Acyclic	No cycles or loops exist. Once a task completes, you never return to it. This prevents infinite loops and ensures the job always terminates.

7.3 DAG in Action — A Hive Example

Consider this query:

```
SELECT department, AVG(salary)
FROM employees
WHERE age > 30
GROUP BY department;
```

The DAG for this query has 4 stages:

Stage	What Happens
Stage 1: Map Task	Read employee data from HDFS files. Apply the WHERE filter (age > 30). Emit only matching records to temporary files.
Stage 2: Shuffle & Sort	Group the filtered records by the 'department' column. Distribute records to the correct reducer nodes.
Stage 3: Reduce Task	Calculate AVG(salary) for each department group. Write the final aggregated result to HDFS.
Stage 4: Final Output	Read results from HDFS. Return the output table to the user through the Driver.

Each stage is a node. Arrows between stages represent dependencies — Stage 2 cannot begin until Stage 1 finishes.

7.4 Why DAG is Powerful

Benefit	Explanation
Clear Dependencies	The DAG makes it explicit which tasks must complete before others can begin — no ambiguity in execution order.
Parallel Execution	If two stages are independent of each other, they can run simultaneously. For example, scanning two different tables at the same time.
Optimization Opportunities	The Optimizer can inspect the DAG and rearrange, merge, or eliminate stages to improve overall efficiency.
Fault Tolerance	If a stage fails, only that stage (and its dependents) needs to be re-executed — not the entire job.

QUICK REFERENCE — Key Terms & Comparisons

YARN vs Hadoop 1.x

Feature	Hadoop 1.x	YARN (Hadoop 2.x/3.x)
Master Daemon	JobTracker	Resource Manager
Worker Daemon	TaskTracker	Node Manager
Resource Unit	Fixed slots	Flexible containers
Max Scale	~4,000 nodes	10,000+ nodes
Frameworks Supported	MapReduce only	MapReduce, Spark, Flink, Tez, etc.
HA Support	No	Yes (ZooKeeper-based)
Single Point of Failure	Yes (JobTracker)	No (RM HA available)
Resource Utilization	Poor (fixed slots)	Excellent (dynamic containers)

YARN Important Ports

Service	Default Port	Purpose
Resource Manager Web UI	8088	Monitor cluster, apps, and queues
Resource Manager	8032	Client-to-RM communication
RM Scheduler	8030	Application Master to RM
Node Manager Web UI	8042	Monitor node containers and resources
Node Manager	8040	Container communication
MapReduce Job History	19888	View completed job history

Hive Component Roles at a Glance

Component	Role	Analogous to
Beeline	CLI for submitting queries	MySQL command line

HiveServer2	Handles concurrent client connections	Web server / API gateway
Driver	Session management, query lifecycle	Request handler
Compiler	Parses and plans query execution	SQL query planner
Optimizer	Improves the execution plan	Query optimizer in RDBMS
Execution Engine	Submits jobs to MapReduce/Tez/Spark	Job dispatcher
Metastore	Stores all table/schema metadata	System catalog / data dictionary
HCatalog	Exposes Hive tables to other tools	Shared data catalog

YARN CLI Reference

```

yarn application -list           # List all running applications
yarn application -kill <app_id> # Kill a specific application
yarn logs -applicationId <app_id> # View application logs
yarn node -list                 # List all cluster nodes
yarn queue -status <queue_name> # View queue status

```